Cache Invalidation Strategies

keeping copies fresh, and why it's so hard

Phase 3 videos on YouTube

> **In one line:** caches stay fresh by TTL expiry (simple, stale up to the TTL), explicit delete on write (fast, but every write path must remember), event-driven invalidation from the database log (nothing forgets), and versioned keys or tag purges; it's hard because reads, writes and invalidation messages race, get delayed or reordered.

1 The famous hard problem

"There are only two hard things in computer science: cache invalidation and naming things" (Phil Karlton). It's an old joke, but it's true. A user changes their name, and three different caches keep showing the old one for hours. Here are the strategies for keeping caches fresh, and why it's hard to get right.



Figure 1. Four ways to invalidate. Real systems combine them: precise invalidation for speed, TTLs as the safety net.

2 TTL-based invalidation

The simplest strategy: don't invalidate at all; entries just **expire** (lesson 19). It's easy and **self-healing**: even if something goes wrong, the stale value disappears eventually. The cost: data can be stale for up to the TTL. Fine for data with a generous staleness budget.

3 Explicit delete on write

When the application updates the database, it deletes the related cache keys, as in cache-aside (lesson 16). It's fast and precise, but **every code path that writes must remember** to invalidate, including admin tools, batch jobs and migrations. And one change can affect many keys: a user's name appears in `user:42`, their posts list, a team page and search results.

4 Event-driven invalidation

Every change to the database is published as an **event**, either by the application or by **change data capture** (CDC), which reads the database's own log (Postgres WAL, MySQL binlog) with a tool like Debezium. Cache invalidators subscribe and delete the affected keys. With CDC, no write path can forget, because the database log captures every change, even a manual SQL fix.

WORKED EXAMPLE: A PROFILE UPDATE

A user updates their profile. The user service writes to Postgres and publishes a `user-updated` event to a Kafka topic (or Debezium emits it from the WAL). Several consumers listen: the profile cache service deletes `user:42`, the feed service clears cached author info, and the search indexer updates the user's document. Each team owns its own invalidation, and the user service doesn't need to know which caches exist.

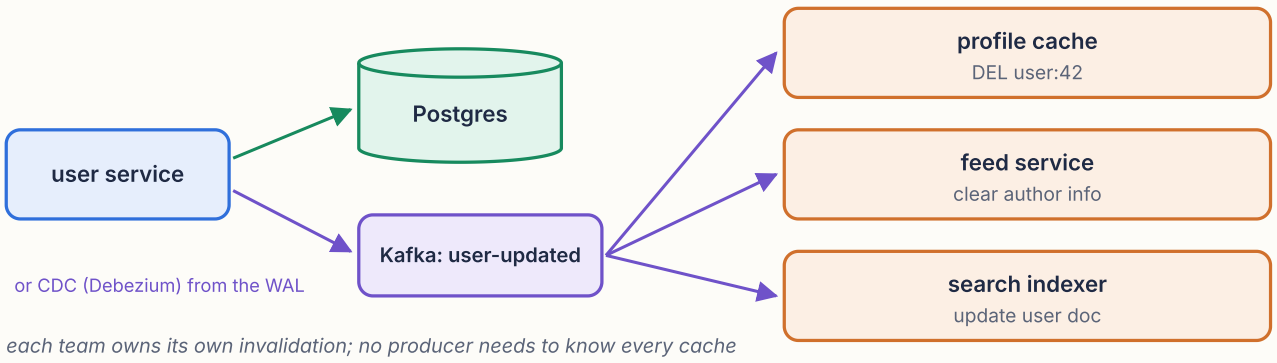


Figure 2. Event-driven invalidation: one change event fans out to every cache that depends on the data.

5 Versioned keys and tag-based purges

- **Versioned keys** put a version in the key: `product:42:v7`. When the product changes, the version increments (stored in the database row, or a tiny version key), and readers ask for the new key. Old versions simply expire. No delete, so no delete race.
- **Tag-based purges** label entries with tags, like `product-42`, and purge every entry with that tag at once. CDNs such as Fastly (surrogate keys) and Cloudflare (cache tags) support it: one product change purges its page, its category pages and its API responses.

6 Why it's hard: distributed timing

Writes and reads race, and invalidation messages can be **delayed, duplicated or reordered**. The classic race: a reader misses and reads the *old* value, then pauses. Meanwhile a writer updates the database and deletes the key. Then the reader wakes up and writes the old value into the cache. The cache now holds stale data until the TTL expires. Next lesson shows how to stop that properly.

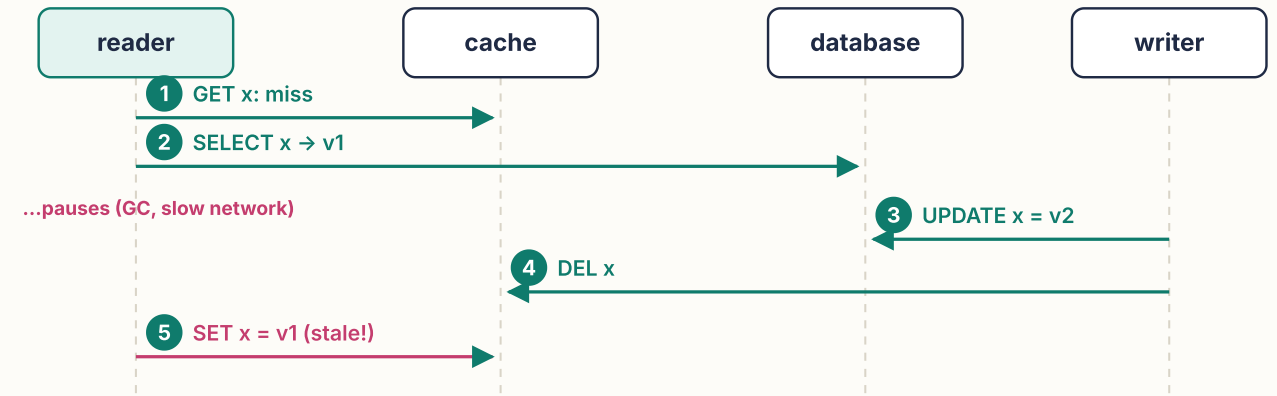


Figure 3. The stale-set race: the delete happens between the reader's database read and its cache write.

COMMON MISTAKES

- Invalidating only the obvious key and forgetting the derived ones (lists, aggregates, rendered pages).
- No TTL as a safety net, so a lost event leaves stale data forever.
- Invalidation consumers that aren't idempotent, or that crash and skip events.
- Long browser or CDN cache lifetimes on data you'll need to change quickly.

7 Key terms

Invalidation: removing or marking a cache entry stale after a change

CDC: change data capture: stream changes from the DB log

Versioned key: the version is part of the key

Stale set: an old value written into the cache after invalidation

Delete on write: the writer deletes related keys

Debezium: a common open-source CDC tool

Tag / surrogate key: label for purging many entries at once

8 Interview questions

Q1. Why is cache invalidation hard? BASIC

Because caches and databases are updated separately, so concurrent reads and writes, delayed or reordered invalidation messages, and many dependent keys can leave stale data behind.

Q2. Describe a race that leaves a stale value. BASIC

A reader misses and reads the old value, a writer updates the database and deletes the key, then the slow reader writes the old value into the cache, where it stays until the TTL.

Q3. Ten services cache user data. How do you invalidate them all when a user changes? SCENARIO

Publish a change event (or use CDC from the database log) to a topic; each service subscribes and invalidates its own keys; keep TTLs as a safety net.

Q4. A product change must update its page, category pages and API responses on the CDN. How? SCENARIO

Tag every cached response with the product's tag (surrogate key) and purge by tag on change; or version the URLs so new content gets new cache keys.

Q5. Why does CDC make invalidation more reliable than app-level deletes? TRICKY

Every committed change appears in the database log, including from batch jobs, admin tools or manual SQL, so no write path can skip invalidation; the app can't forget.

Q6. How do versioned keys avoid the delete race? TRICKY

Readers always look up the current version first, so a slow reader that caches an old value writes it under the old key, which nobody reads anymore; it simply expires.

Next: SD-F-009 · Cache Consistency